Assignment II: Training a MiniGPT

Dunjie Lu

dunjielu1219@connect.hku.hk

Released 29 October 2026 · Due 22 November 2026, 23:59 HKT

1 Language Modeling

- Build a small decoder-only Transformer that predicts the next token.
- Train from scratch on a prepared subset of short English stories.
- Use the trained model to continue a prompt, one token at a time.

Illustrative next-token example (not model output)

Input: “Once upon a” → Target: “time”

1.1 What will you learn

- Implement causal self-attention and a Transformer block in PyTorch.
- Train, save and reload a language model on a single GPU.
- Compare model and optimization changes using controlled experiments.
- Interpret validation loss, perplexity and generated text.

2 Working on the Assignment

- Download the starter ZIP and fixed data ZIP; no pretrained weights are needed.
- Complete four methods in src/model.py, then run public tests and a smoke run.
- Train the baseline and two controlled experiments from random initialization.
- Package the code, one checkpoint, three runs of evidence and a short report.

Use the codebase and shell scripts. No notebook is required.

2.1 Environment and Fixed Data

1. Extract starter, then extract the data ZIP inside its root directory.

2. Use Python 3.12/3.13; install hardware-compatible PyTorch 2.10.0.

```
python3.13 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python scripts/check_setup.py
```

The data/ folder contains train.bin, val.bin, tokenizer.json, manifest.json and license files. Keep the fixed data and tokenizer unchanged.

CPU for tests/smoke; BF16-capable CUDA GPU for the baseline. MPS is not supported.

2.2 Assignment Tasks

- Task 1: Implement masked attention, MLP, residual block and generation.
- Task 2: Train the unchanged baseline for 2,500 optimizer updates.
- Task 3: Run one model-side and one optimization-side experiment.
- Task 4: Submit report.pdf (up to 2 pages) with analysis and continuations.

Complete the baseline and both controlled experiments; submit one selected checkpoint.

2.3 Task 1: Implement the Model

Method in src/model.py	What to complete
CausalSelfAttention.forward	Q/K/V, head splitting, causal mask, projection
MLP.forward	Linear → GELU → Linear → dropout
Block.forward	Pre-normalization and residual connections
GPT.generate	Context crop, temperature/top-k, EOS sampling

Provided: embeddings, language-model head, loss, training loop and logging.

2.3 Implementation Checks

- Finish all four TODOs before running model tests or training.
- Check causal masking, output shapes, gradients and prompt preservation.
- A 20-update smoke run checks the workflow; it is not the full baseline.

```
python -m pytest -q
bash scripts/run_smoke.sh cpu
# Or use cuda on a compatible GPU
```

Fresh starter tests intentionally fail until TODOs are complete. Public tests are not exhaustive.

2.4 Task 2: Train the Baseline

Setting	Baseline: configs/baseline.yaml
Model	6 layers · 6 heads · embedding width 384
Tokenizer / context	8,192-token vocabulary · context 256
Parameters / optimizer	13,891,584 (tied embeddings) · AdamW
Batch / training	16 sequences · accumulation 1 · 2,500 updates
Effective batch / budget	4,096 tokens per update · 10,240,000 tokens

Keep the baseline YAML unchanged. Runtime is informational, not graded.

2.4 Training and Run Outputs

bash scripts/train_baseline.sh cuda

Equivalent command (append --resume only for an interrupted run):

```
python -m src.train --config configs/baseline.yaml \
  --out_dir outputs/baseline --device cuda
```

- Use a new output directory for every new run; existing nonempty paths are protected.
- best.pt is selected by public validation loss. last.pt stores resume state and must not be submitted.

Resume only the same run/configuration. Each experiment starts from random initialization.

2.5 Task 3: Controlled Experiments

Run	Change from baseline	Keep fixed
Baseline	None	Released configuration
Model-side	Example: 6 → 4 layers	Optimizer, data, seed, budget
Optimization-side	Example: peak learning rate	Model, data, seed, budget

Three full training runs in total, all starting from random initialization.

Keep vocabulary, context and 10.24M-token budget fixed. Use the supported GPT family (depth/width/heads), at most 20M parameters.

2.5 What Makes a Good Experiment?

- State a hypothesis and change one main factor at a time.
- Compare learning curves, validation loss, parameter count and runtime.
- Explain negative results: improvement over the baseline is not required.
- A fixed token budget is not a fixed compute budget. Discuss the trade-off.
- A single-seed result is limited evidence; avoid claims of statistical significance.

2.6 Generation and Evaluation

Run for each experiment (replace outputs/baseline with its directory):

```
python scripts/plot_run.py outputs/baseline
python -m src.sample --checkpoint outputs/baseline/best.pt \
  --out outputs/baseline/samples.json
python -m src.evaluate --checkpoint outputs/baseline/best.pt \
  --tokens data/val.bin --max-tokens 0 \
  --out outputs/baseline/evaluation.json
```

Sampling defaults: temperature 0.8, top-k 40, seed 2026, up to 160 new tokens.
Use the supplied prompts consistently; discuss coherence and failure cases.

Full public evaluation uses FP32; trainer validation is sampled/mixed precision.
Do not compare these as the same protocol. Perplexity = exp(cross-entropy).

2.7 Task 4: Final Report

- Method: explain your model implementation and next-token objective.
- Experiments: compare the baseline and two controlled runs in one table, with learning curves and the evaluation protocol.
- Generation: show representative continuations and discuss limitations.
- Reproducibility: put exact commands and environment details in README.md.

report.pdf: up to 2 pages. Full generated samples belong in each run’s samples.json.

3 Package and Check Your Submission

Run from the starter root after all three runs and report.pdf are ready:

```
python scripts/package_submission.py --student-id YOUR_STUDENT_ID \
  --baseline outputs/baseline --model-experiment outputs/your_model_run \
  --optimizer-experiment outputs/your_optimizer_run --select baseline \
  --report report.pdf --out submission.zip
bash scripts/check_submission.sh submission.zip
```

Replace YOUR_STUDENT_ID and the run paths. Choose --select from: baseline, model_experiment or optimizer_experiment.

Rename the checked ZIP to your student-ID filename before uploading.

3 Submission ZIP Structure

student-ID.zip (files at the ZIP root)

Folder / file	Contents
submission.json / report.pdf	Manifest / report (up to 2 pages)
best.pt	One selected checkpoint, matching the manifest
src/ scripts/ configs/ tests/	Completed code, helpers, configs and tests
runs/<role>/	baseline, model_experiment, optimizer_experiment
Root support files	README.md, requirements.txt, LICENSE, pytest.ini

Upload one ZIP (maximum 100 MB). No data, environments, caches or last.pt.
Checker safety limits: 200 MiB unpacked; 150 MiB checkpoint.

4 Important Dates and Submission

- Release and submissions open: 29 October 2026.
- Deadline: Sunday, 22 November 2026, 23:59 Hong Kong time (UTC+8).
- One final submission: check the ZIP before clicking Submit assignment.
- Late penalty: up to 1 day 10%; up to 2 days 20%; up to 7 days 50%; more than 7 days 100%.

Late submissions close on 29 November 2026 at 23:59 HKT.
A saved draft is not a final submission. Verify the submitted status in Moodle.

5 Marking Scheme

Category	Marks
Code and model performance	80
Report and reproducibility	20
Total	100

Code/model: correctness, reproducible baseline and controlled runs, training evidence, and performance on unseen text.

Report: clear methods, justified comparisons, learning curves, representative generations and limitations. Maximum 2 pages.

5 Required Run Evidence

Each runs/<role>/ directory contains these eight files:

metrics.json
environment.json
eval.jsonl
loss_curve.png
config.json
train.csv
samples.json
evaluation.json

The packaging script collects them from your three run directories.
Select the final checkpoint using public validation only.

Passing the format checker is not a grade or proof of training provenance.

6 Important Notice

- Train from scratch using only the supplied training split and fixed tokenizer.
- Implement explicit attention; do not replace it with a complete attention module.
- Preserve public method signatures, checkpoint keys and supported GPT architecture.
- Do not warm-start an experiment from another run’s weights.
- Check the code, model and report before making your one final submission.

Questions!

Please contact Dunjie Lu

dunjielu1219@connect.hku.hk